Deep Learning in Computer Vision· Lecture 6 of 9

ResNet, Transfer Learning and Vision Transformers

Skip connections, reusing pretrained models, and a first look at attention

Where We Are, and Today's Roadmap

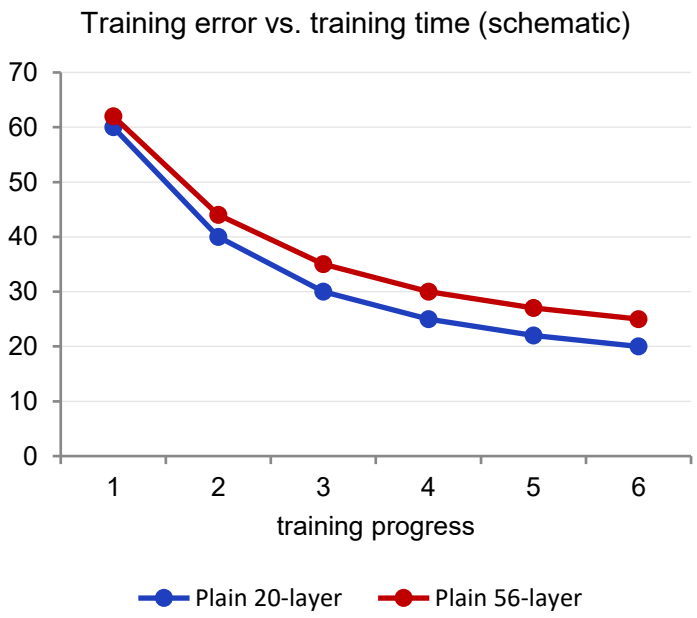
- **Lecture 5**: VGG and GoogLeNet showed that **deeper** networks (more stacked conv + ReLU layers; **depth** = number of weight layers) recognise images better.
- **Natural question**: can we keep adding layers forever? With plain stacking the answer is **no**, and today's first topic explains why and how **ResNet** fixes it.
    - **Part A, ResNet** (skip connections): the idea that made 50-152 layer networks trainable.
    - **Part B, Transfer learning**: reuse a network already trained on millions of images instead of training from scratch.
    - **Part C, Vision Transformers (ViT)**: a different architecture that replaces convolution with **attention**.
- **Goal**: you can explain $y = F(x) + x$, decide when to freeze or fine-tune, and trace an image through a ViT.

Glossary: Terms Used in This Lecture

Term	Meaning
Training error	Fraction of training images classified wrongly: how well the optimizer fits the data it sees
Test / validation error	Error on unseen images: measures generalization (Lecture 3)
Overfitting	Low training error but high test error: the model memorized the training set
Optimization problem	Good weights exist, but gradient descent fails to find them (training error stays high)
Mapping H(x)	The function a group of layers computes: output as a function of its input x
Identity mapping	H(x) = x: the output equals the input; the layers "do nothing"
Backbone	The feature-extracting part of a network (the conv stages), reused for many tasks
Pretrained / fine-tuning	Weights already trained (usually on ImageNet) / continuing their training on new data
Token	One vector fed to a transformer (a word in text, an image patch in ViT)
A plain network	An ordinary stack of layers with no skip connections. Each layer feeds only the next one: conv → ReLU → conv → ReLU

The Degradation Problem

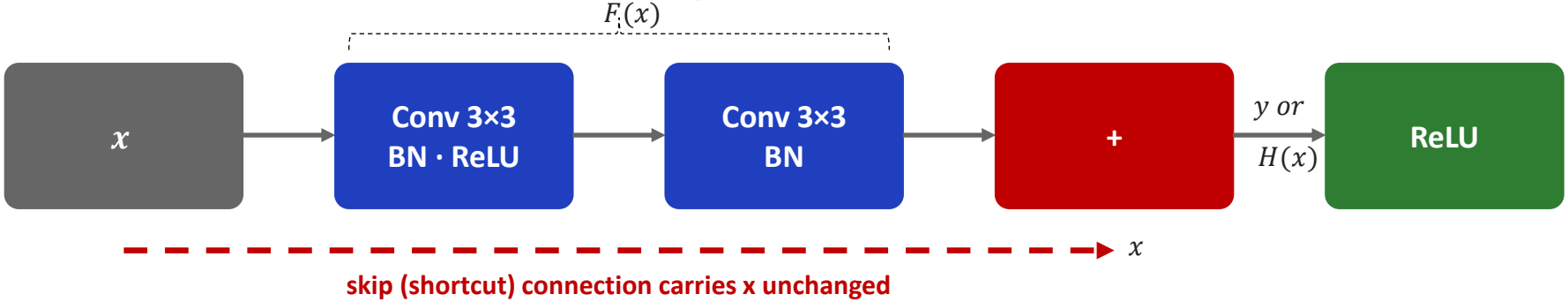
- **Experiment** (He et al., 2015, Deep Residual Learning for Image Recognition): train a plain (no skip connections) 20-layer and 56-layer CNN on CIFAR-10 (benchmark dataset).
- **Result**: the 56-layer net had **higher training error** (and higher test error) than the 20-layer net.
- **Why it is surprising**: the deeper net could simply copy the shallow net and let the extra layers do nothing (identity), so it should be **at least as good**.
- **Why it is not overfitting**: overfitting means low training error but high test error. Here even **training** error is worse.
- **Conclusion**: It is an **optimization problem**. Plain deep network stacks are hard for gradient descent to train, even when a good solution exists.



Why Is "Doing Nothing" or Identity Mapping So Hard?

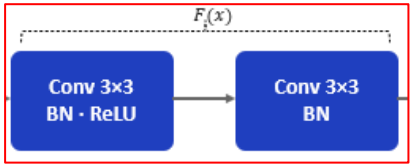
- A group of layers computes a mapping **H(x)** from its input **x**. To "do nothing" it must learn **H(x) = x** (identity).
- For a plain block (conv → ReLU → conv) the weights must be set **exactly right**, and ReLU sets every negative value to 0, destroying information that the identity needs to keep.
- With random initial weights the block starts **far** from identity, and gradient descent has to find the exact weights through a long chain of layers.
- Deeper plain nets therefore add many layers that **damage** the signal instead of passing it on.
- **ResNet's idea**: make "doing nothing" the **easy default** by giving the block a direct shortcut for x.

The Residual Block: Diagram and Symbols



Symbol / term	What it means
x	Input to the block: a feature map (e.g. 56×56×64)
$F(x)$	The block's two conv layers: what the layers learn
$y = H(x) = F(x) + x$	Output = learned part plus the unchanged input
$+$	Element-wise addition: same position, same channel; x and F(x) must have the same shape
$BN, ReLU$	Batch normalization (for stability) and ReLU; ReLU is applied after the addition

Residual Learning: Learn the Difference



- Let $H(x)$ = the mapping we ideally want the block to compute.
- Plain block: layers try to learn $H(x)$ directly.
- **Residual block**: layers learn $F(x) = H(x) - x$, and the output is $F(x) + x = H(x)$.
- **Residual** = what is left over: the small correction that must be added to the input.
- **Analogy**: editing a draft vs. writing from scratch. If the input is 10 and the target is 10.2, a plain layer must produce 10.2; a residual layer only learns **+0.2**.
- If the best behaviour is identity, the layers just push $F(x) \rightarrow 0$ (all weights → 0), which is easy.

Key shift: from learning a whole function to learning a small correction.

Why It Helps: The Gradient View

- **Backprop** (Lecture 2) sends the loss gradient backward through every layer, multiplying by each layer's derivative.
- Plain stack: many numbers < 1 multiplied together → gradient → 0 (**vanishing gradient**; Lecture 3: $0.25^{10} \approx 10^{-6}$).
- Residual block: $y = F(x) + x$, so $\boldsymbol{\partial y/\partial x = \partial F/\partial x + 1}$.
- Chain rule:: $\frac{\partial L}{\partial x} = \frac{\partial L}{\partial y} \times \frac{\partial y}{\partial x} = \frac{\partial L}{\partial y} \times (\frac{\partial F}{\partial x} + 1)$.
- The "+1" is the **skip path**: gradient passes through it multiplied by exactly **1**, however small ∂F/∂x is.
- So gradients reach the first layers of a 100-layer net almost undiminished.

Symbol / term	What it means
L	Loss (Lecture 2)
$\partial L/\partial x$	How the loss changes if x changes: the gradient we need
$\partial F/\partial x$	Derivative of the conv layers: often small
$+1$	Derivative of the shortcut (y has a term x)

Why Residual Networks Work: Four Views

Easy identity

If a block is not needed, it learns $F(x) \approx 0$ and passes x through. Adding layers can **not hurt** much.

Gradient highway

The skip path gives gradients a **direct route** from the loss to early layers (previous slide).

Implicit ensemble

A net with n blocks has **many paths** (each block used or skipped). Deleting one block barely changes accuracy, like an ensemble of shallower nets.

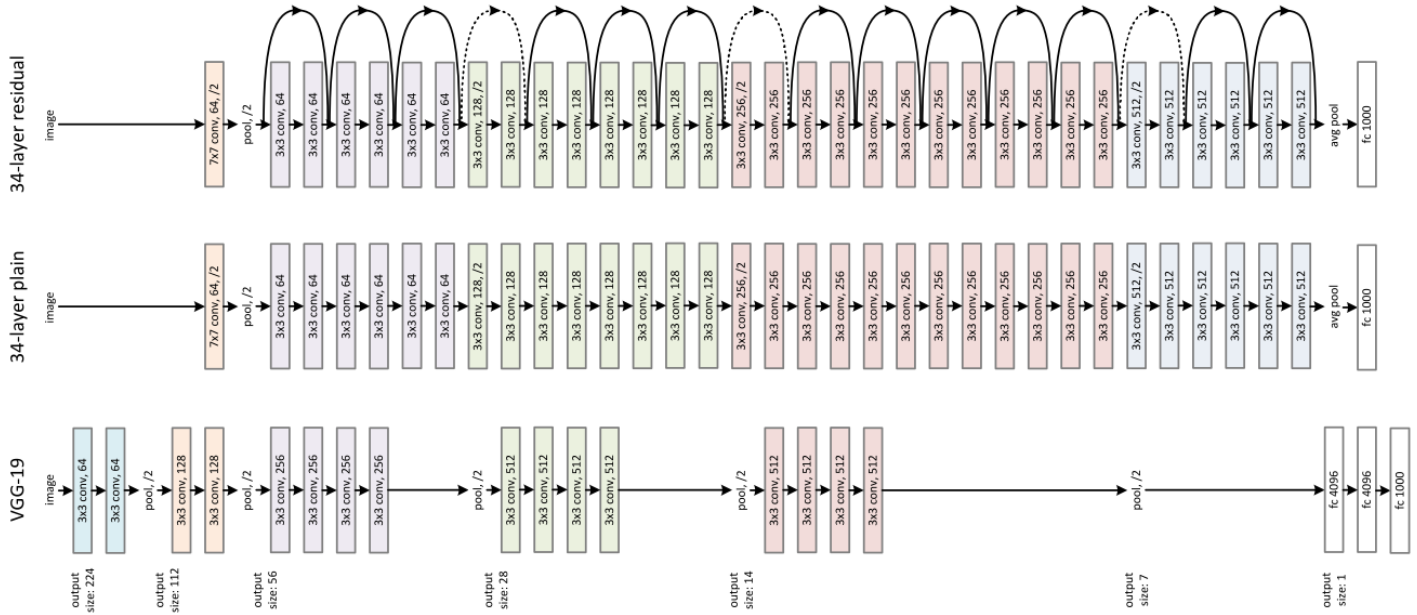
Smoother loss

skips make it **smoother**, so gradient descent works better.

When Shapes Differ: Identity vs Projection Shortcut

- Addition needs **equal shapes**.
    - Addition is element-wise, so both paths must have the same shape
    - Between stages ResNet halves height/width (**stride 2**) and doubles the channels, so x no longer matches $F(x)$.
- **Identity shortcut** (shapes equal): $y = F(x) + x$. No parameters. Used in most blocks.
- **Projection shortcut** (shapes differ): $y = F(x) + \boldsymbol{W_s}\, \boldsymbol{x}$, where
    - $\boldsymbol{W_s}$ is a **1×1 convolution with stride 2**;
        - it resizes x to match $F(x)$.

ResNet Architecture



- conv1 7×7, stride of 2 (\2) → max pool → **4 stages** of residual blocks (64, 128, 256, 512 filters) → global avg pool → FC 1000
- Each stage halves the size and doubles the channels

Basic Block vs Bottleneck Block

Basic block (ResNet-18/34)

3×3 conv → 3×3 conv
e.g., Two or multiple layers, simple; used in shallower models.

Bottleneck (ResNet-50/101/152)

1×1 (reduce 256→64) → **3×3** (64) → **1×1** (expand 64→256)
Same cost as basic, but 3 layers → **deeper**, cheaper.

Reuses the 1×1 bottleneck idea from Inception.

The ResNet Family

Model	Layers	Params	GFLOPs	Block
ResNet-18	18	11.7M	1.8	Basic
ResNet-34	34	21.8M	3.6	Basic
ResNet-50	50	25.6M	3.8	Bottleneck
ResNet-101	101	44.5M	7.6	Bottleneck
ResNet-152	152	60.2M	11.3	Bottleneck

- Note:
  - ResNet-152 is 8× deeper than VGG-16 with fewer parameters.
  - ResNet-50 is the most common backbone.

Impact of ResNet, and What Came After

- **Result**: 1st place in ILSVRC 2015 classification with **3.57% top-5 error** (top-5 error = true class not among the 5 highest-scored guesses); also won detection and COCO detection/segmentation tracks.
- **Skip connections became standard**:
    - U-Net and FCN (Lecture 7),
    - YOLO backbones (Lecture 9) and **every transformer block** (comes latter in these slides).

- **What Came After**
    - **Wide ResNet**: fewer but wider blocks.
    - **ResNeXt**: parallel grouped convolutions inside a block.

Beyond ResNet: Other Efficient Designs (Self-Read)

DenseNet

Each layer receives the **concatenated** outputs of **all** earlier layers. Strong **feature reuse**, fewer parameters; high memory use.

MobileNet

Depthwise separable convolution: a 3×3 conv per channel, then a 1×1 conv to mix channels. About 8-9× fewer operations; made for phones.

EfficientNet

Compound scaling: grow depth, width and input resolution together with one coefficient instead of only depth.

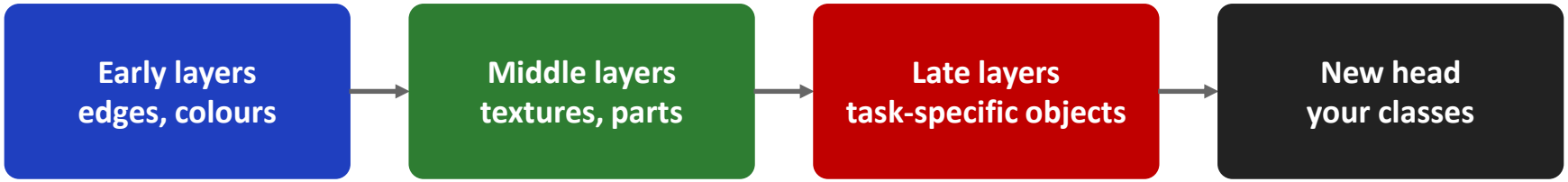
ConvNeXt

A ResNet **modernized** with transformer-era choices (larger kernels, LayerNorm, GELU): shows CNNs still compete.

Transfer Learning: The Idea

- **Training from scratch** needs millions of labeled images and days of GPU time.
- **ImageNet** (1.2M images, 1000 classes) already taught networks to see. A model trained on it is **pretrained**.
- **Transfer learning** = start from those pretrained weights and adapt them to **your** task (a few hundred or thousand images): e.g. X-ray, defects, plants.
- **Why it works**: early layers detect **generic** things (edges, colours, textures) useful for any image; only the final layers are specific to ImageNet classes.
- **Target domain** = the images of your new task. The more similar it is to ImageNet, the better the transfer.

What Do Layers Learn? Which Part to Reuse?



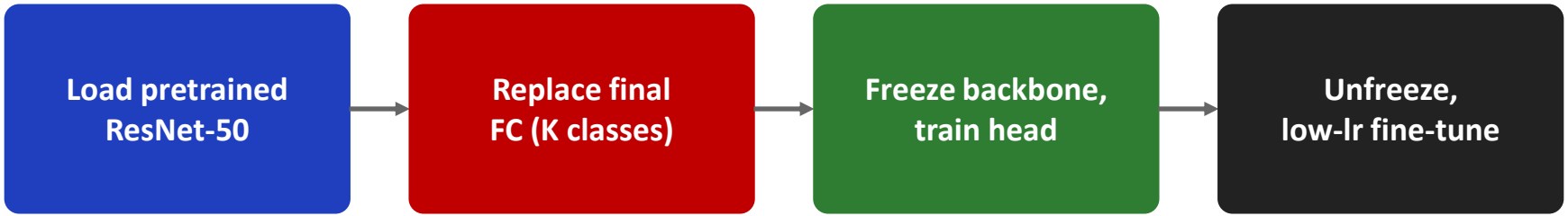
- **Early layers: generic** → almost always reuse, safe to freeze.
- **Middle layers**: transferable if the new images look natural.
- **Late layers: specific** to the original 1000 classes → most worth retraining.
- **Head** (the last FC layer): must be **replaced**, since your task has K classes, not 1000.
- **Freezing** a layer = its weights are not updated during training.

Feature Extraction vs. Fine-Tuning

	Feature extraction	Fine-tuning
What is trained	Only the new head	New head plus some or all backbone layers
Backbone	Frozen (fixed features)	Unfrozen, small learning rate
Data needed	Little (hundreds)	More (thousands+)
Cost, risk	Fast, low overfitting risk	Slower, can overfit or forget useful features
Choose when	Few images, similar to ImageNet	More images or a different domain (medical, satellite)

- **Rule of thumb**: small data → freeze more; large data or different domain → unfreeze more.
- **Learning rate**: head ≈ 10^{-3}, backbone ≈ 10^{-4}-10^{-5} (small, so pretrained knowledge is refined, not erased).

A Practical Recipe



- Use the **same input size and normalization** as pretraining
- Add **augmentation**; monitor validation loss; use **Adam** or SGD + momentum
- Handle class imbalance (Lecture 3) and report precision/recall

Practical Recipe and Code (Detailed) – Self-Read

- 1. Load pretrained backbone; **replace the final FC** with K outputs.
- 2. **Freeze** backbone; train the head a few epochs.
- 3. **Unfreeze** (all or the last stages); train with a small learning rate.
- 4. Use the **same preprocessing** as pretraining: resize to 224, normalize with ImageNet mean (0.485, 0.456, 0.406) and std (0.229, 0.224, 0.225).
- 5. Add augmentation, watch validation loss, handle class imbalance (Lecture 3).
- 6. report precision/recall

```
import torch.nn as nn, torchvision
m = torchvision.models.resnet50(
  weights="IMAGENET1K_V2")
for q in m.parameters():
    q.requires_grad = False  # freeze
m.fc = nn.Linear(2048, K)  # new head
# only m.fc is trained now
```

Why Transformers for Vision?

- A conv layer sees only a small **local neighbourhood** (3×3). The **receptive field** (region of the input a neuron depends on, Lecture 4) grows only slowly with depth.
- Relating two distant parts of an image needs **many layers**.
- **Transformer** (Vaswani et al., 2017, "Attention Is All You Need"):
  - built for text, where any word can depend on any other word.
  - It uses **self-attention**: every element looks directly at every other element in one step.
- **Token** = one element of the input sequence (a word; in vision, an image patch).
- **ViT** (Dosovitskiy et al., 2020) cuts an image into patches and treats them as tokens.

Attention: The Intuition

- **Text example**: "The animal did not cross the street because **it** was too tired." To understand "it", the model should look at **"animal"**.
    - Self-attention learns such links.
- **Image example**: a patch on a dog's ear should look at the patches of the dog's head and legs, not at the sky.
- Each token produces **three vectors**, like a **search engine**:
    - **Query (Q)**: "what am I looking for?"
    - **Key (K)**: "what do I contain?" (compared with every query) - give a **similarity score**.
    - **Value (V)**: "the information I hand over" if I am relevant.
- Output for a token = **weighted average of all Values**, with softmax weights = how well its Query matches each Key.

Vision Transformer (ViT) Pipeline



- 224×224 image → **14×14 = 196 patches** (+1 class token = 197 tokens)
- **MLP**: two fully connected layers applied to each token separately (Lecture 1), with GELU (Gaussian Error Linear Unit) activation.
- Position embeddings tell the model **where** each patch came from
- ViT-Base: 12 layers, 12 heads, ≈86M parameters
- The [CLS] output is used for classification

The **[CLS] token** (short for **Classification token**) is a special learnable embedding prepended to the input sequence in Vision Transformers (ViTs) and NLP Transformer models to serve as a **global representation aggregator for classification tasks**.

CNN vs Vision Transformer

	CNN	ViT
Inductive bias (built-in assumptions)	Locality + weight sharing (Lecture 4): good for small data	Almost none: must learn spatial structure from data
Data needed	Works with small / medium datasets	Needs large datasets or pretraining
Receptive field	Grows layer by layer	Global from layer 1 (every patch sees all patches)
Compute vs image size	Grows linearly with pixels	Grows with N^2 (N = number of patches)
Strength	Efficient, strong on small data, edge devices	Scales well with data and model size

Variants and Trends

- **DeiT**: trains ViT well on ImageNet-1k alone, using strong augmentation and **distillation** (learning from a teacher CNN).
- **Swin Transformer**: attention inside **shifted local windows**, giving a hierarchy of feature maps like a CNN; good for detection and segmentation.
- **Hybrids**: a small conv stem makes the patch embeddings, then a transformer.
- **Foundation models (transformer-based)**: very large models pretrained once, reused for many tasks.
    - **CLIP** links images and text;
    - **SAM** (Segment Anything) segments any object from a prompt.
- **In practice**: CNNs stay strong for small data and real-time devices; **transformers** dominate large-scale pretraining.

Summary

- **CNN blocks** (Lecture 4): conv, ReLU, BN, pooling, FC.
- **VGG** (small 3×3 filters, depth) and **Inception** (multi-scale, 1×1 bottleneck): Lecture 5.
- **ResNet**: $y = F(x) + x$ lets 50-152 layers train, by easing identity and preserving gradients.
- **Transfer learning**: reuse ImageNet-pretrained backbones; freeze or fine-tune depending on data.
- **ViT**: patches → tokens → self-attention; strong with large-scale pretraining.
- **Next week**: segmentation and detection (U-Net, YOLO), which reuse backbones and skip connections.